COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010-Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 1, 2026	**DATE SUBMITTED:**	Oct 1, 2026
NAME:	Vince Joseph Garcia	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. _10.1__ [Graphs]

1. PROCEDURE OUTPUT

ILOA

Header File

```
#ifndef GRAPH_H
#define GRAPH_H

//stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

//structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};
class DiaGraph {
    //insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode (int value, int weight, adjNode* head){
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;

        newNode->next = head; // point new node to current head

    return newNode;
    }
    int N; //number of nodes in the graph

public: //adjacency list as array of pointers
    adjNode **head;      //adjacency list as array of pointers
    //constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        // allocate new node
        head = new adjNode*[N]();
        this->N = N;
```

```
34          // initialize head pointer for all vertices
35          for (int i = 0; i < N; ++i)
36          head[i] = nullptr;
37          // construct directed graph by adding edges to it
38          for (unsigned i = 0; i < n; i++) {
39              int start_ver = edges[i].start_ver;
40              int end_ver = edges[i].end_ver;
41              int weight = edges[i].weight;
42              // insert in the beginning
43              adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
44              // point head pointer to new node
45              head[start_ver] = newNode;
46          }
47      }
48      // Destructor
49      ~DiaGraph() {
50          for (int i = 0; i < N; i++)
51          delete[] head[i];
52          delete[] head;
53      }
54  };
55
56  #endif
```

Main cpp

```cpp
#include <iostream>
#include "graph.h"

void display_AdjList(adjNode* ptr, int i){
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
            << ", " << ptr->cost << ") ";

        ptr = ptr->next;
    }
    std::cout << std::endl;
}

int main(){
    // graph edges array.
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
    };
    int N = 6; // Number of vertices in the graph
    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);
    // construct graph
    DiaGraph diagraph(edges, n, N);
    // print adjacency list representation of graph
    std::cout<<"Graph adjacency list "<<std::endl
        <<"(start_vertex, end_vertex,weight):"<<std::endl;

    for (int i = 0; i < N; i++)
    {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }
    return 0;
}
```

Output

```
Graph adjacency list
(start_vertex, end_vertex,weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

ILOB1

Header

```cpp
#ifndef GRAPH_1_H
#define GRAPH_1_H

#include <string>
#include <vector>
#include <iostream>

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T>& e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T>& e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream& operator<<(std::ostream& os, const Graph<T>& G) {
    for (size_t i = 1; i < G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto& e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

template <typename T>
class Graph {
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N) {
    }
```

```cpp

    // Return number of vertices in the graph
    auto vertices() const {
        return V;
    }

    // Return all edges in the graph
    auto& edges() const {
        return edge_list;
    }

    void add_edge(Edge<T>&& e) {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto& e : edge_list) {
            if (e.src == v) {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    // Overloads the << operator so a graph can be written directly to a strea
    template <typename U>
    friend std::ostream& operator<<(std::ostream& os, const Graph<U>& G);

private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};

#endif
```

Main file

```cpp
#include <iostream>
#include <vector>
#include <set>
#include <map>
#include <stack>
#include "graph1.h"

template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto& i : edges)
        for (auto& j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}

template <typename T>
auto depth_first_search(const Graph<T>& G, size_t dest) {
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1

    while (!stack.empty()) {
        auto current_vertex = stack.top();
        stack.pop();
```

```cpp
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex)) {
                // If the vertex hasn't been visited, insert it in the stack.
                if (visited.find(e.dest) == visited.end()) {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

template <typename T>
void test_DFS() {
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}

int main() {
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

Output

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

ILOB2

Header

```cpp
#ifndef GRAPH_2_H
#define GRAPH_2_H

#include <string>
#include <vector>
#include <iostream>

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T>& e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T>& e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream& operator<<(std::ostream& os, const Graph<T>& G) {
    for (size_t i = 1; i < G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto& e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os; // the sample forgot this line
}

```

```cpp
template <typename T>
class Graph {
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const {
        return V;
    }

    auto& edges() const {
        return edge_list;
    }

    void add_edge(Edge<T>&& e) {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto& e : edge_list) {
            if (e.src == v) {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    // renamed to U so it does not clash with the class template parameter T
    template <typename U>
    friend std::ostream& operator<<(std::ostream& os, const Graph<U>& G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

#endif
```

Main file

```cpp
#include <iostream>
#include <vector>
#include <set>
#include <map>
#include <queue>
#include "graph2.h"

template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto& i : edges)
        for (auto& j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

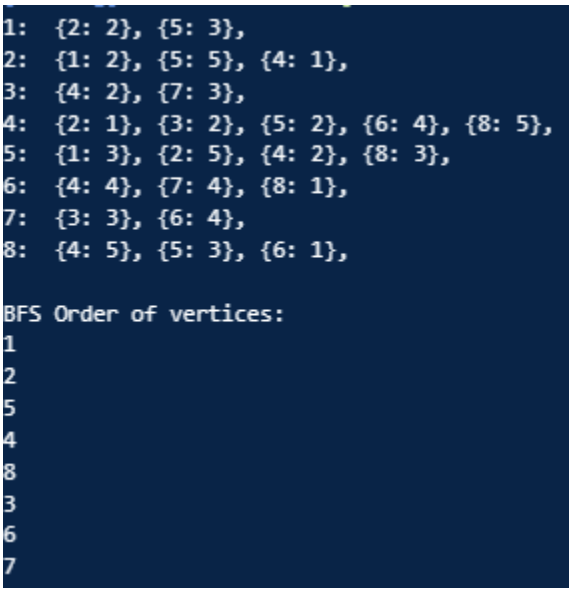
    return G;
}

template <typename T>
auto breadth_first_search(const Graph<T>& G, size_t dest) {
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1

```

```
36      while (!queue.empty()) {
37          auto current_vertex = queue.front();
38          queue.pop();
39
40          // If the current vertex hasn't been visited in the past
41          if (visited.find(current_vertex) == visited.end()) {
42              visited.insert(current_vertex);
43              visit_order.push_back(current_vertex);
44
45              for (auto e : G.outgoing_edges(current_vertex))
46                  queue.push(e.dest);
47          }
48      }
49      return visit_order;
50  }
51
52  template <typename T>
53  void test_BFS() {
54      // Create an instance of and print the graph
55      auto G = create_reference_graph<unsigned>();
56      std::cout << G << std::endl;
57
58      // Run BFS starting from vertex ID 1 and print the order
59      // in which vertices are visited.
60      std::cout << "BFS Order of vertices: " << std::endl;
61      auto bfs_visit_order = breadth_first_search(G, 1);
62      for (auto v : bfs_visit_order)
63          std::cout << v << std::endl;
64  }
65
66  int main() {
67      using T = unsigned;
68      test_BFS<T>();
69      return 0;
```

Output



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

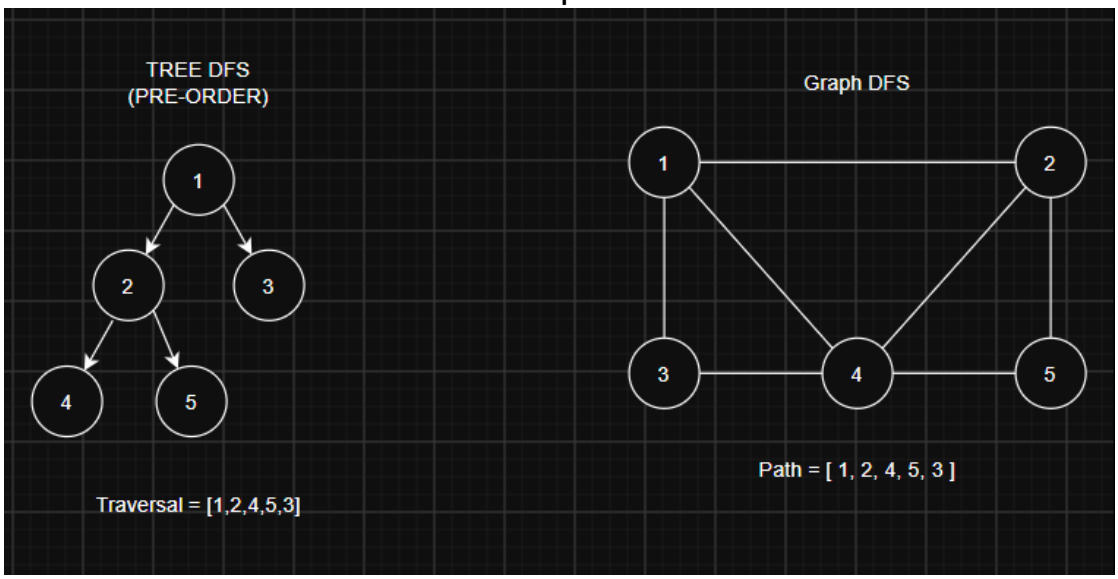
1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

- DFS or depth-first search would be the most optimal for this because dfs goes through a graph by exploring along a single path as deep as possible before backtracking which suits the person's goal to explore a complete route to its end before backtracking onto alternative unvisited locations.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

- The direct equivalent of DFS in tree traversal strategies is Pre-order traversal because Pre-order visits the root node first and goes straight down the deepest left path before backtracking to visit the right branch, following the exact same deep-first behavior as DFS.

Graph



PseudoCode

```
ALGORITHM UNIFIED_DFS(node_or_vertex v)
   IF (v IS NULL OR v IS VISITED)
      RETURN

   VISIT(v)

   IF (STRUCTURE IS GRAPH)
      MARK v AS VISITED

   FOR EACH neighbor_or_child u ADJACENT TO v DO
      IF (u IS NOT VISITED)
         CALL UNIFIED_DFS(u)
```

CODE

```cpp
#include <iostream>
#include <vector>
#include <set>

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

void treeDFS(TreeNode* root) {
    if (root == nullptr) return;
    std::cout << root->val << " ";
    treeDFS(root->left);
    treeDFS(root->right);
}

struct Graph {
    int V;
    std::vector<std::vector<int>> adj;
    Graph(int V) : V(V), adj(V) {}

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
};

void graphDFSHelper(const Graph& G, int v, std::set<int>& visited) {
    if (visited.find(v) != visited.end()) return;

    std::cout << v << " ";
    visited.insert(v);

```

```
36      for (int u : G.adj[v]) {
37          if (visited.find(u) == visited.end()) {
38              graphDFSHelper(G, u, visited);
39          }
40      }
41  }
42
43  void graphDFS(const Graph& G, int start) {
44      std::set<int> visited;
45      graphDFSHelper(G, start, visited);
46  }
47
48  int main() {
49      // Construct 5-Node Tree
50      TreeNode* root = new TreeNode(1);
51      root->left = new TreeNode(2);
52      root->right = new TreeNode(3);
53      root->left->left = new TreeNode(4);
54      root->left->right = new TreeNode(5);
55
56      std::cout << "Tree DFS Output: ";
57      treeDFS(root);
58      std::cout << std::endl;
59
60      // Construct 5-Node Graph
61      Graph g(6);
62      g.addEdge(1, 2);
63      g.addEdge(1, 3);
64      g.addEdge(2, 4);
65      g.addEdge(2, 5);
66      g.addEdge(4, 3);
67
68      std::cout << "Graph DFS Output: ";
69      graphDFS(g, 1);
70      std::cout << std::endl;
71
72      return 0;
73  }
```

Output

```
Tree DFS Output: 1 2 4 5 3
Graph DFS Output: 1 2 4 3 5
```

3. In the performed code, what data structure is used to implement the Breadth First Search?

- The code puts the starting node into the queue and loops while the queue still has nodes inside. It pops the front node out, prints it, grabs all its adjacent neighbors, marks them as visited, and pushes them to the back of the queue. Because new nodes go to the back of the queue, the code processes every neighbor on the current layer first before moving to nodes on the next level down.

4. How many times can a node be visited in the BFS?

- The code keeps track of every node it touches using a visited set. When the loop looks at a node's adjacent neighbors, it runs an IF check against that set. If a node is already in visited, the code skips it entirely so it never gets added to the queue again or processed twice.

Conclusion

In this activity, I learned how to represent graphs using adjacency matrices and lists, as well as how to perform Depth-First Search (DFS) and Breadth-First Search (BFS) traversals in C++. The procedure clearly demonstrated how swapping a stack for a queue changes the execution from a deep, path-by-path search to a layer-by-layer exploration, while using a visited set keeps the code from getting stuck in infinite loops when handling cyclic graphs. Analyzing the supplementary activity reinforced these concepts by showing that tree traversals are just acyclic versions of graph DFS that skip visited tracking, and that map exploration with backtracking naturally aligns with DFS. Overall, I performed well in successfully writing the code and getting the expected traversal outputs, though my main areas for improvement are getting better at manual memory cleanup when using dynamic pointers and practicing paper tracing for larger graph networks.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction and spell check**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.